
13 More Issues In Input/Output

- Using *argc* and *argv*
- Detecting Errors in Reading/Writing
- Standard I/O Devices
- I/O Redirection
 - Redirecting the Output
 - Redirecting the Input
 - Both Ways at Once
- Summary
- Exercise

In Chapters 11 and 12 we saw how Console I/O and File I/O are done in C. There are still some more issues related with input/output that remain to be understood. These issues help in making the I/O operations more elegant.

Using *argc* and *argv*

To execute the file-copy programs that we saw in Chapter 12 we are required to first type the program, compile it, and then execute it. This program can be improved in two ways:

- (a) There should be no need to compile the program every time to use the file-copy utility. It means the program must be executable at command prompt (A> or C> if you are using MS-DOS, Start | Run dialog if you are using Windows and \$ prompt if you are using Unix).
- (b) Instead of the program prompting us to enter the source and target filenames, we must be able to supply them at command prompt, in the form:

```
filecopy PR1.C PR2.C
```

where, PR1.C is the source filename and PR2.C is the target filename.

The first improvement is simple. In MS-DOS, the executable file (the one which can be executed at command prompt and has an extension .EXE) can be created in Turbo C/C++ by using the key F9 to compile the program. In VC++ compiler under Windows same can be done by using F7 to compile the program. Under Unix this is not required since in Unix every time we compile a program we always get an executable file.

The second improvement is possible by passing the source filename and target filename to the function **main()**. This is illustrated below:

```
#include "stdio.h"
main ( int argc, char *argv[ ] )
{
    FILE *fs, *ft ;
    char ch ;

    if ( argc != 3 )
    {
        puts ( "Improper number of arguments" ) ;
        exit( ) ;
    }

    fs = fopen ( argv[1], "r" ) ;
    if ( fs == NULL )
    {
        puts ( "Cannot open source file" ) ;
        exit( ) ;
    }

    ft = fopen ( argv[2], "w" ) ;
    if ( ft == NULL )
    {
        puts ( "Cannot open target file" ) ;
        fclose ( fs ) ;
        exit( ) ;
    }

    while ( 1 )
    {
        ch = fgetc ( fs ) ;

        if ( ch == EOF )
            break ;
        else
            fputc ( ch, ft ) ;
    }
}
```

```
    fclose ( fs );  
    fclose ( ft );  
}
```

The arguments that we pass on to **main()** at the command prompt are called command line arguments. The function **main()** can have two arguments, traditionally named as **argc** and **argv**. Out of these, **argv** is an array of pointers to strings and **argc** is an **int** whose value is equal to the number of strings to which **argv** points. When the program is executed, the strings on the command line are passed to **main()**. More precisely, the strings at the command line are stored in memory and address of the first string is stored in **argv[0]**, address of the second string is stored in **argv[1]** and so on. The argument **argc** is set to the number of strings given on the command line. For example, in our sample program, if at the command prompt we give,

filecopy PR1.C PR2.C

then,

argc would contain 3

argv[0] would contain base address of the string "filecopy"

argv[1] would contain base address of the string "PR1.C"

argv[2] would contain base address of the string "PR2.C"

Whenever we pass arguments to **main()**, it is a good habit to check whether the correct number of arguments have been passed on to **main()** or not. In our program this has been done through,

```
if ( argc != 3 )  
{  
    printf ( "Improper number of arguments" ) ;  
    exit( ) ;  
}
```

Rest of the program is same as the earlier file-copy program. This program is better than the earlier file-copy program on two counts:

- (a) There is no need to recompile the program every time we want to use this utility. It can be executed at command prompt.
- (b) We are able to pass source file name and target file name to **main()**, and utilize them in **main()**.

One final comment... the **while** loop that we have used in our program can be written in a more compact form, as shown below:

```
while ( ( ch = fgetc ( fs ) ) != EOF )  
    fputc ( ch, ft );
```

This avoids the usage of an indefinite loop and a **break** statement to come out of this loop. Here, first **fgetc (fs)** gets the character from the file, assigns it to the variable **ch**, and then **ch** is compared against **EOF**. Remember that it is necessary to put the expression

```
ch = fgetc ( fs )
```

within a pair of parentheses, so that first the character read is assigned to variable **ch** and then it is compared with **EOF**.

There is one more way of writing the **while** loop. It is shown below:

```
while ( !feof ( fs ) )  
{  
    ch = fgetc ( fs );  
    fputc ( ch, ft );  
}
```

Here, **feof()** is a macro which returns a 0 if end of file is not reached. Hence we use the **!** operator to negate this 0 to the truth value. When the end of file is reached **feof()** returns a non-zero

value, ! makes it 0 and since now the condition evaluates to false the **while** loop gets terminated.

Note that in each one of them the following three methods for opening a file are same, since in each one of them, essentially a base address of the string (pointer to a string) is being passed to **fopen()**.

```
fs = fopen ( "PR1.C" , "r" );
fs = fopen ( filename, "r" );
fs = fopen ( argv[1] , "r" );
```

Detecting Errors in Reading/Writing

Not at all times when we perform a read or write operation on a file are we successful in doing so. Naturally there must be a provision to test whether our attempt to read/write was successful or not.

The standard library function **ferror()** reports any error that might have occurred during a read/write operation on a file. It returns a zero if the read/write is successful and a non-zero value in case of a failure. The following program illustrates the usage of **ferror()**.

```
#include "stdio.h"
main()
{
    FILE *fp ;
    char ch ;

    fp = fopen ( "TRIAL", "w" ) ;

    while ( !feof ( fp ) )
    {
        ch = fgetc ( fp ) ;
        if ( ferror() )
        {
```

```
        printf ( "Error in reading file" );
        break ;
    }
    else
        printf ( "%c", ch );
}

fclose ( fp );
}
```

In this program the **fgetc()** function would obviously fail first time around since the file has been opened for writing, whereas **fgetc()** is attempting to read from the file. The moment the error occurs **ferror()** returns a non-zero value and the **if** block gets executed. Instead of printing the error message using **printf()** we can use the standard library function **perror()** which prints the error message specified by the compiler. Thus in the above program the **perror()** function can be used as shown below.

```
if ( ferror() )
{
    perror ( "TRIAL" );
    break ;
}
```

Note that when the error occurs the error message that is displayed is:

TRIAL: Permission denied

This means we can precede the system error message with any message of our choice. In our program we have just displayed the filename in place of the error message.

Standard I/O Devices

To perform reading or writing operations on a file we need to use the function **fopen()**, which sets up a file pointer to refer to this file. Most OSs also predefine pointers for three standard files. To access these pointers we need not use **fopen()**. These standard file pointers are shown in Figure 13.1

Standard File pointer	Description
stdin	standard input device (Keyboard)
stdout	standard output device (VDU)
stderr	standard error device (VDU)

Figure 13.1

Thus the statement **ch = fgetc (stdin)** would read a character from the keyboard rather than from a file. We can use this statement without any need to use **fopen()** or **fclose()** function calls.

Note that under MS-DOS two more standard file pointers are available—**stdprn** and **stdaux**. They stand for standard printing device and standard auxiliary device (serial port). The following program shows how to use the standard file pointers. It reads a file from the disk and prints it on the printer.

```
/* Prints file contents on printer */
#include "stdio.h"
main()
{
    FILE *fp ;
    char ch ;
```



```
fp = fopen ( "poem.txt", "r" );

if ( fp == NULL )
{
    printf ( "Cannot open file" );
    exit( ) ;
}

while ( ( ch = fgetc ( fp ) ) != EOF )
    fputc ( ch, stdprn ) ;

fclose ( fp ) ;
}
```

The statement **fputc (ch, stdprn)** writes a character read from the file to the printer. Note that although we opened the file on the disk we didn't open **stdprn**, the printer. Standard files and their use in redirection have been dealt with in more details in the next section.

Note that these standard file pointers have been defined in the file "stdio.h". Therefore, it is necessary to include this file in the program that uses these standard file pointers.

I/O Redirection

Most operating systems incorporate a powerful feature that allows a program to read and write files, even when such a capability has not been incorporated in the program. This is done through a process called 'redirection'.

Normally a C program receives its input from the standard input device, which is assumed to be the keyboard, and sends its output to the standard output device, which is assumed to be the VDU. In other words, the OS makes certain assumptions about where input

should come from and where output should go. Redirection permits us to change these assumptions.

For example, using redirection the output of the program that normally goes to the VDU can be sent to the disk or the printer without really making a provision for it in the program. This is often a more convenient and flexible approach than providing a separate function in the program to write to the disk or printer. Similarly, redirection can be used to read information from disk file directly into a program, instead of receiving the input from keyboard.

To use redirection facility is to execute the program from the command prompt, inserting the redirection symbols at appropriate places. Let us understand this process with the help of a program.

Redirecting the Output

Let's see how we can redirect the output of a program, from the screen to a file. We'll start by considering the simple program shown below:

```
/* File name: util.c */
#include "stdio.h"<+>
main()
{
    char ch ;
    while ( ( ch = getc ( stdin ) ) != EOF )
        putc ( ch, stdout ) ;
}
```

On compiling this program we would get an executable file UTIL.EXE. Normally, when we execute this file, the **putc()** function will cause whatever we type to be printed on screen, until we don't type Ctrl-Z, at which point the program will terminate, as

shown in the following sample run. The Ctrl-Z character is often called end of file character.

```
C>UTIL.EXE
perhaps I had a wicked childhood,
perhaps I had a miserable youth,
but somewhere in my wicked miserable past,
there must have been a moment of truth ^Z
C>
```

Now let's see what happens when we invoke this program from in a different way, using redirection:

```
C>UTIL.EXE > POEM.TXT
C>
```

Here we are causing the output to be redirected to the file POEM.TXT. Can we prove that this the output has indeed gone to the file POEM.TXT? Yes, by using the TYPE command as follows:

```
C>TYPE POEM.TXT
perhaps I had a wicked childhood,
perhaps I had a miserable youth,
but somewhere in my wicked miserable past,
there must have been a moment of truth
C>
```

There's the result of our typing sitting in the file. The redirection operator, '>', causes any output intended for the screen to be written to the file whose name follows the operator.

Note that the data to be redirected to a file doesn't need to be typed by a user at the keyboard; the program itself can generate it. Any output normally sent to the screen can be redirected to a disk file. As an example consider the following program for generating the ASCII table on screen:

```
/* File name: ascii.c*/
main()
{
    int ch ;

    for ( ch = 0 ; ch <= 255 ; ch++ )
        printf ( "\n%d %c", ch, ch ) ;
}
```

When this program is compiled and then executed at command prompt using the redirection operator,

```
C>ASCII.EXE > TABLE.TXT
```

the output is written to the file. This can be a useful capability any time you want to capture the output in a file, rather than displaying it on the screen.

DOS predefines a number of filenames for its own use. One of these names is PRN, which stands for the printer. Output can be redirected to the printer by using this filename. For example, if you invoke the “ascii.exe” program this way:

```
C>ASCII.EXE > PRN
```

the ASCII table will be printed on the printer.

Redirecting the Input

We can also redirect input to a program so that, instead of reading a character from the keyboard, a program reads it from a file. Let us now see how this can be done.

To redirect the input, we need to have a file containing something to be displayed. Suppose we use a file called NEWPOEM.TXT containing the following lines:

Let's start at the very beginning,
A very good place to start!

We'll assume that using some text editor these lines have been placed in the file NEWPOEM.TXT. Now, we use the input redirection operator '<' before the file, as shown below:

```
C>UTIL.EXE < NEWPOEM.TXT
Let's start at the very beginning,
A very good place to start!
C>
```

The lines are printed on the screen with no further effort on our part. Using redirection we've made our program UTIL.C perform the work of the TYPE command.

Both Ways at Once

Redirection of input and output can be used together; the input for a program can come from a file via redirection, at the same time its output can be redirected to a file. Such a program is called a filter. The following command demonstrates this process.

```
C>UTIL < NEWPOEM.TXT > POETRY.TXT
```

In this case our program receives the redirected input from the file NEWPOEM.TXT and instead of sending the output to the screen it would redirect it to the file POETRY.TXT.

Similarly to send the contents of the file NEWPOEM.TXT to the printer we can use the following command:

```
C>UTIL < NEWPOEM.TXT > PRN
```

While using such multiple redirections don't try to send output to the same file from which you are receiving input. This is because

the output file is erased before it's written to. So by the time we manage to receive the input from a file it is already erased.

Redirection can be a powerful tool for developing utility programs to examine or alter data in files. Thus, redirection is used to establish a relationship between a program and a file. Another OS operator can be used to relate two programs directly, so that the output of one is fed directly into another, with no files involved. This is called 'piping', and is done using the operator '|', called pipe. We won't pursue this topic, but you can read about it in the OS help/manual.

Summary

- (a) We can pass parameters to a program at command line using the concept of 'command line arguments'.
- (b) The command line argument **argv** contains values passed to the program, whereas, **argc** contains number of arguments.
- (c) We can use the standard file pointer **stdin** to take input from standard input device such as keyboard.
- (d) We can use the standard file pointer **stdout** to send output to the standard output device such as a monitor.
- (e) We can use the standard file pointers **stderr** and **stderr** to interact with printer and auxiliary devices respectively.
- (f) Redirection allows a program to read from or write to files at command prompt.
- (g) The operators < and > are called redirection operators.

Exercise

[A] Answer the following:

- (a) How will you use the following program to
 - Copy the contents of one file into another.
 - Print a file on the printer.
 - Create a new file and add some text to it.

- Display the contents of an existing file.

```
#include "stdio.h"
main()
{
    char ch, str[10];
    while ( ( ch = getc ( stdin ) ) != -1 )
        putc ( ch, stdout );
}
```

(b) State True or False:

1. We can send arguments at command line even if we define **main()** function without parameters.
2. To use standard file pointers we don't need to open the file using **fopen()**.
3. Using **stdaux** we can send output to the printer if printer is attached to the serial port.
4. The zeroth element of the **argv** array is always the name of the exe file.

(c) Point out the errors, if any, in the following program

```
main ( int ac, char ( *) av[ ] )
{
    printf ( "\n%d", ac );
    printf ( "\n%s", av[0] );
}
```

[B] Attempt the following:

(a) Write a program to carry out the following:

- (a) Read a text file provided at command prompt
- (b) Print each word in reverse order

For example if the file contains

INDIA IS MY COUNTRY

Output should be

AIDNI SI YM YRTNUOC

- (b) Write a program using command line arguments to search for a word in a file and replace it with the specified word. The usage of the program is shown below.

C> change <old word> <new word> <filename>

- (c) Write a program that can be used at command prompt as a calculating utility. The usage of the program is shown below.

C> calc <switch> <n> <m>

Where, **n** and **m** are two integer operands. **switch** can be any one of the arithmetic or comparison operators. If arithmetic operator is supplied, the output should be the result of the operation. If comparison operator is supplied then the output should be **True** or **False**.